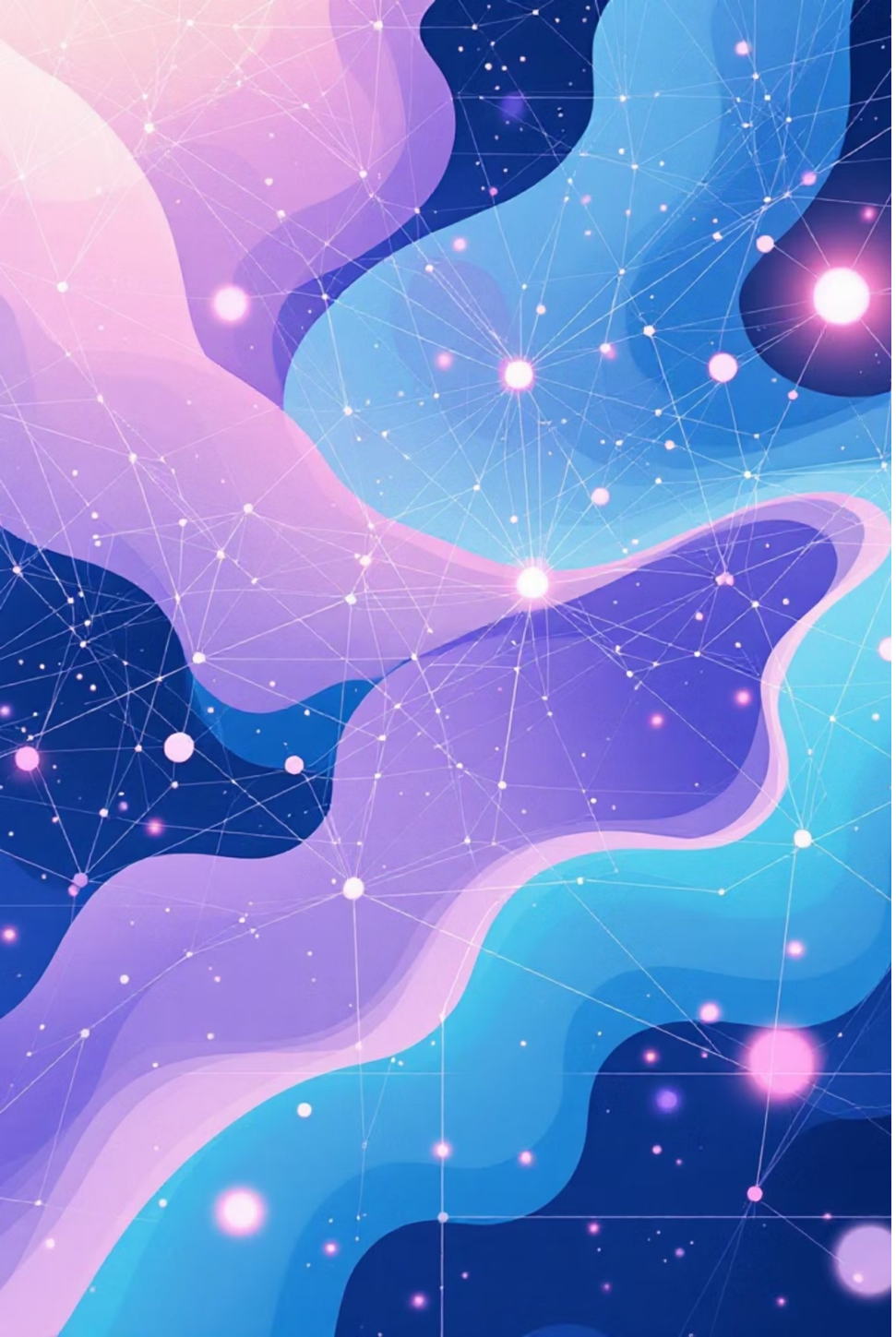


Graph Traversal: BFS and DFS Explained

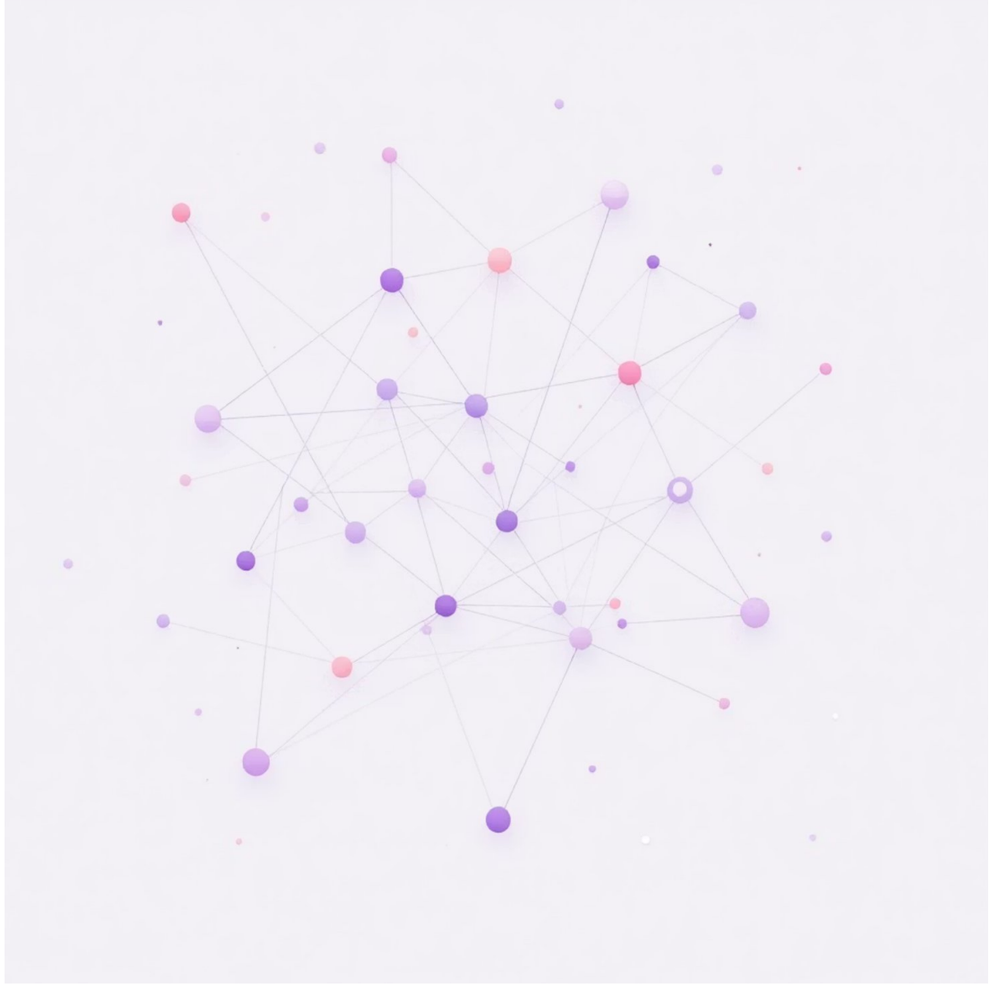
Explore the fundamental algorithms for navigating the intricate world of data structures: Breadth-First Search (BFS) and Depth-First Search (DFS).

Prepared By :
Afsana Begum
Assistant Professor
Daffodil International University

What is a Graph?

A graph is a non-linear data structure comprising a set of interconnected entities known as **vertices** (or nodes) and the links between them, called **edges**.

- Graphs can be **directed** (edges have a specific flow direction) or **undirected** (edges allow bidirectional movement).
- They can also be **weighted** (edges have associated costs or distances) or **unweighted**.
- Graph traversal is the process of visiting all nodes in a graph in a systematic manner to ensure no node is missed.



Introduction to BFS (Breadth-First Search)

Level by Level Exploration

BFS systematically explores a graph by visiting all the neighbors of a node before moving on to the next level of neighbors.

Queue Data Structure

It utilizes a **Queue** (First-In, First-Out or FIFO) to keep track of the nodes that need to be visited next, ensuring a breadth-first approach.

Shortest Path Finder

BFS is particularly effective in unweighted graphs for finding the **shortest path** between a starting node and any other reachable node.

How BFS Works (Step-by-Step)

01

Initialize

Begin at the designated source node, marking it as visited to avoid redundant processing.

02

Enqueue Source

Add the source node to the queue, initiating the traversal process.

03

Process Queue

While the queue is not empty, dequeue a node and process it.

04

Explore Neighbors

Enqueue all unvisited adjacent nodes of the currently processed node, marking them as visited.

05

Continue Traversal

Repeat the process until all reachable nodes have been successfully visited and the queue is empty. If a vertex has not been visited, call the DFS function starting from that vertex. This ensures that every connected component is explored entirely.

Introduction to DFS (Depth-First Search)

Deep Exploration

DFS explores each branch of a graph as deeply as possible before backtracking to explore other branches.

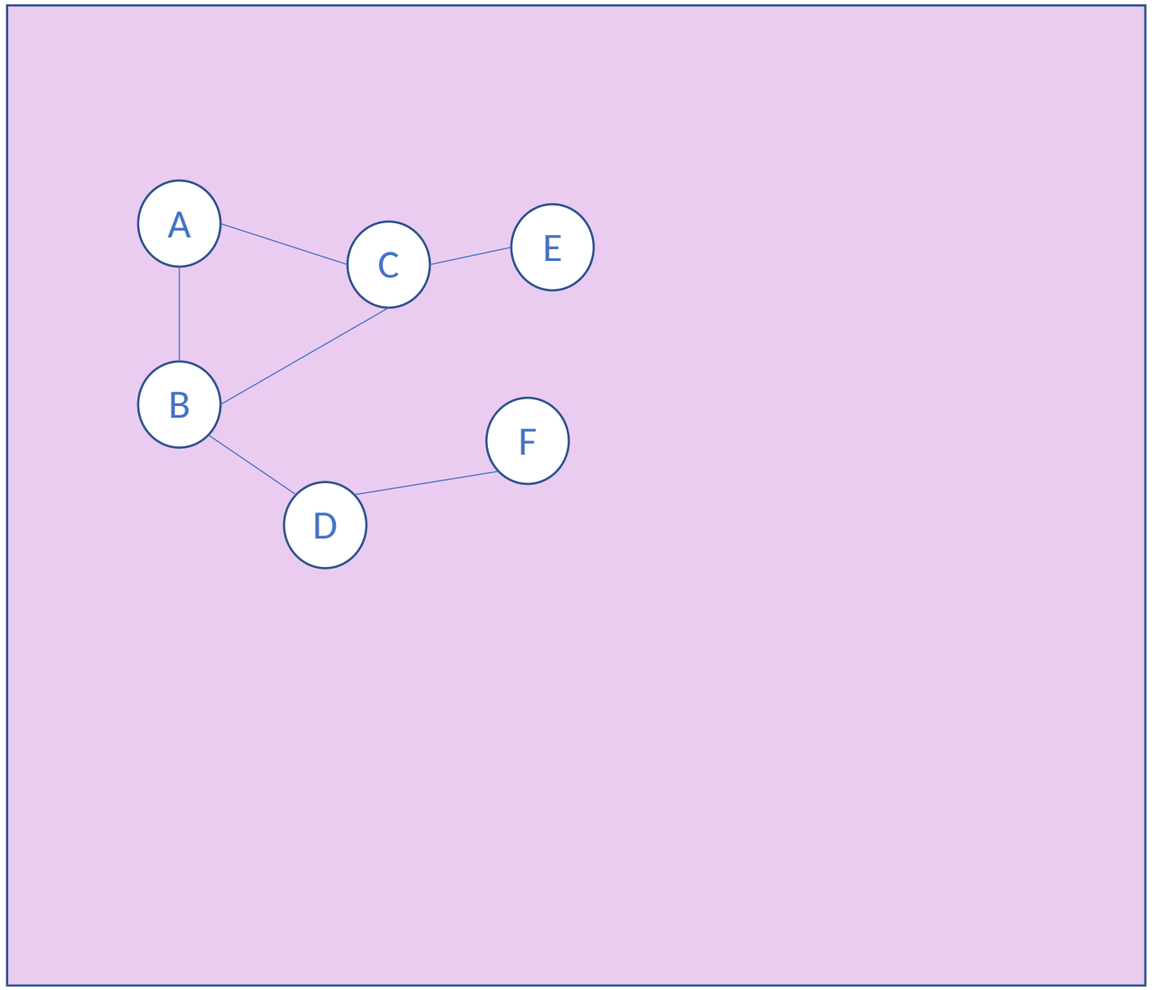
Stack or Recursion

It typically uses a **Stack** (Last-In, First-Out or LIFO) or recursive function calls to manage the nodes to visit. If a vertex has not been visited, call the DFS function starting from that vertex. This ensures that every connected component is explored entirely.

Versatile for Paths

DFS is highly effective for tasks like pathfinding, detecting cycles within a graph, and performing topological sorting.

Example Graph for Traversal



We'll use this graph to trace through both BFS and DFS algorithms step-by-step.

BFS Step-by-Step: Queue Visualization

01

Step 1: Start Traversal

`Queue: [A]` → Visit A, add neighbors B, C

02

Step 2: Process Node A

`Queue: [B, C]` → Visit B, add neighbor D

03

Step 3: Process Node B

`Queue: [C, D]` → Visit C, add neighbor E

04

Step 4: Process Node C

`Queue: [D, E]` → Visit D, add neighbor F

05

Step 5: Process Node D

`Queue: [E, F]` → Visit E

06

Step 6: Process Node E

`Queue: [F]` → Visit F

07

Step 7: Process Node F

`Queue: []` → Done

DFS Step-by-Step: Stack Visualization

01

Step 1: Start Traversal

`Stack: [A]` → Visit A, push neighbor B

02

Step 2: Process Node A

`Stack: [B]` → Visit B, push neighbor D

03

Step 3: Process Node B

`Stack: [D]` → Visit D, push neighbor F

04

Step 4: Process Node D

`Stack: [F]` → Visit F, no unvisited neighbors, pop

05

Step 5: Backtrack to D

`Stack: []` → Backtrack to D, pop

06

Step 6: Backtrack to B

`Stack: []` → Backtrack to B, push neighbor C

07

Step 7: Process Node B (continued)

`Stack: [C]` → Visit C, push neighbor E

08

Step 8: Process Node C

`Stack: [E]` → Visit E, no unvisited neighbors, pop

09

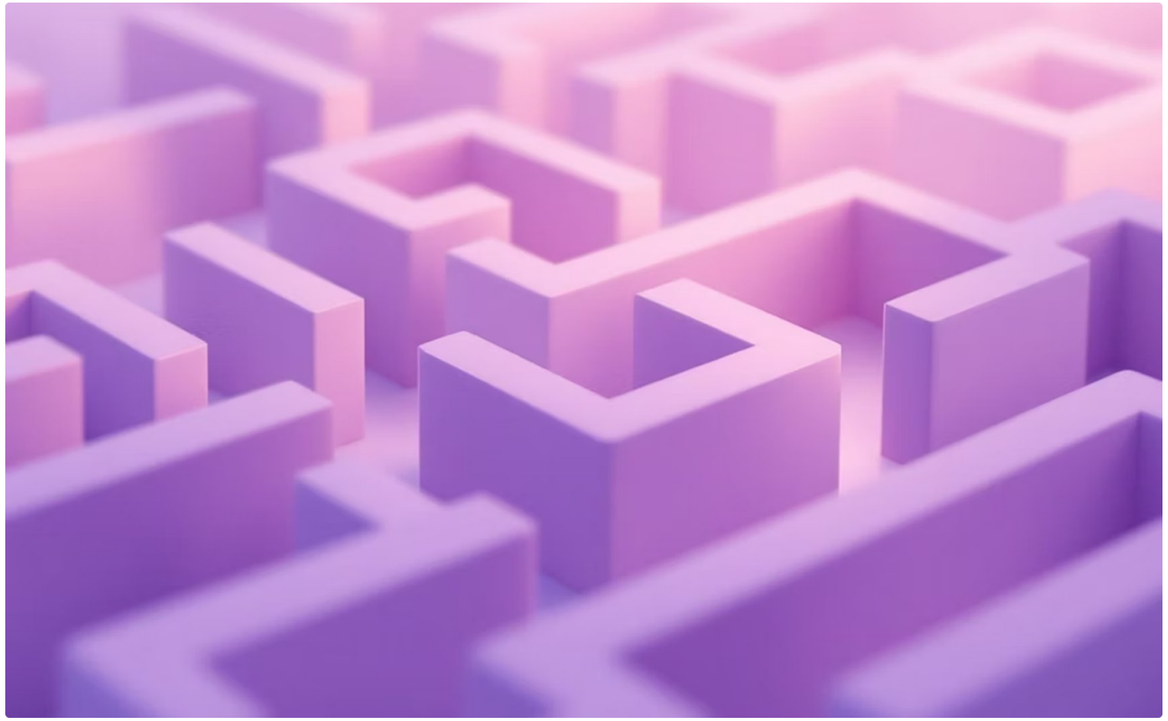
Step 9: Finish Traversal

`Stack: []` → Done

BFS vs DFS: Key Differences

Data Structure	Queue (FIFO)	Stack (LIFO) or Recursion
Traversal Pattern	Level by level, explores breadth first	Goes deep along one path before backtracking
Use Cases	Shortest path in unweighted graphs, finding closest nodes	Pathfinding, cycle detection, topological sorting, connected components
Memory Usage	Can be high for graphs with many nodes at a single level	Generally lower, but can be high for deep, narrow graphs due to recursion stack

Summary & Applications



BFS Applications

- Finding the shortest path in unweighted graphs
- Social networking for finding connections
- Web crawlers to index pages
- Bipartite graph checking

DFS Applications

- Cycle detection in graphs
- Topological sorting of dependencies
- Solving mazes and puzzles
- Finding strongly connected components

Mastering BFS and DFS is crucial for developing advanced graph algorithms and solving complex problems. Practice these foundational algorithms to enhance your data structure skills!